Senior DevOps / SRE Interview Questions & Answers — Part 3

Continuation of advanced DevOps / SRE / Platform Engineering interview preparation.

These answers are written for Lead/Staff-level interviews where interviewers expect:

- production debugging
- architecture thinking
- reliability mindset
- operational maturity
- incident leadership

---

71. How do you debug DNS failures inside Kubernetes?

Answer

DNS issues are very common in Kubernetes.

My approach:

Step 1 — Validate pod DNS resolution

```
kubectl exec -it <pod> -- nslookup service-name
```

Step 2 — Check CoreDNS health

```
kubectl get pods -n kube-system
```

Step 3 — Validate service discovery

Check:

- service name
- namespace
- cluster domain

Step 4 — Analyze network policies

Sometimes DNS traffic gets blocked.

Step 5 — Check node-level DNS

Possible causes:

- upstream resolver failure
- VPC DNS issue
- CoreDNS overload

At scale, I monitor:

- CoreDNS latency
- request rate
- cache hit ratio

---

72. What happens during a rolling update internally?

Answer

Kubernetes gradually replaces old pods with new pods.

Flow:

1. New ReplicaSet created
2. New pods scheduled
3. Readiness probes validated
4. Old pods terminated gradually

Controlled by:

```
maxSurge
maxUnavailable
```

Important production considerations:

- graceful shutdown
- connection draining
- backward compatibility
- database migrations

Improper rolling updates can create cascading failures.

---

73. How do you debug failed readiness probes?

Answer

I first determine whether:

- application unhealthy
- probe configuration incorrect

Checks:

```
kubectl describe pod
```

Then verify:

- endpoint path
- timeoutSeconds
- initialDelaySeconds
- startup duration
- dependency readiness

Many applications fail readiness because dependencies initialize slowly.

I tune probes based on real startup metrics.

---

74. What is the difference between requests and limits?

Answer

Requests determine scheduling guarantees.

Limits define maximum allowed usage.

Example:

- request = guaranteed minimum
- limit = hard cap

Important behaviors:

- CPU throttling happens at limits
- memory exceeding limit causes OOMKill

Poor resource tuning leads to:

- unstable clusters
- wasted cost
- noisy neighbors

---

75. How do you troubleshoot Kubernetes networking issues?

Answer

I debug layer by layer.

Step 1 — Validate pod connectivity

```
kubectl exec -it <pod> -- ping
```

Step 2 — Validate services/endpoints

```
kubectl get svc
kubectl get endpoints
```

Step 3 — Validate CNI health

- Calico
- Cilium
- Flannel

Step 4 — Check network policies

Step 5 — Trace packet flow

Client → Ingress → Service → Pod

Step 6 — Validate DNS separately

Networking problems often appear as application failures.

76. Explain Kubernetes taints and tolerations.

Answer

Taints repel pods from nodes.

Tolerations allow pods to schedule onto tainted nodes.

Use cases:

- dedicated workloads
- GPU nodes
- infra isolation
- spot instances

Example:

```
kubectl taint nodes node1 key=value:NoSchedule
```

I use taints heavily for workload isolation in production.

---

77. How do you debug ingress issues?

Answer

I validate the full traffic path.

Step 1 — Check ingress rules

```
kubectl describe ingress
```

Step 2 — Validate ingress controller

Check:

- logs
- health
- backend mappings

Step 3 — Validate service endpoints

Step 4 — Validate TLS certificates

Step 5 — Test backend directly

I isolate whether failure occurs at:

- DNS
- LB
- ingress
- service
- application

---

78. What causes ImagePullBackOff?

Answer

Most common causes:

- incorrect image
- missing tag
- registry auth failure
- network issue
- image deleted

My checks:

```
kubectl describe pod
```

Then validate:

- imagePullSecrets
- registry access
- IAM permissions
- DNS/networking

At enterprise scale, registry availability becomes critical.

---

79. How do you manage Kubernetes upgrades safely?

Answer

I upgrade incrementally.

Process:

1. Review compatibility matrix
2. Test in lower environments
3. Upgrade control plane
4. Upgrade worker nodes gradually
5. Validate workloads
6. Monitor cluster health

Critical checks:

- deprecated APIs
- CNI compatibility
- CSI compatibility
- ingress compatibility

I always maintain rollback plans.

---

80. Explain PodDisruptionBudget.

Answer

PDB prevents excessive voluntary disruptions.

Example:

```
minAvailable: 2
```

Protects workloads during:

- node drains
- upgrades
- autoscaling

Without PDBs, rolling operations can accidentally create outages.

---

81. How do you investigate high API server latency?

Answer

Possible causes:

- etcd latency
- excessive API calls
- large clusters
- controller overload
- webhook slowness

I investigate:

- API server metrics
- etcd health
- audit logs
- controller activity

At scale, poorly designed operators can overload the API server.

---

82. Explain GitOps and its advantages.

Answer

GitOps uses Git as the source of truth.

Flow:

1. Change committed to Git
2. GitOps controller detects change
3. Cluster reconciles automatically

Advantages:

- auditability
- rollback simplicity
- drift detection
- declarative operations
- better governance

Tools:

- ArgoCD
- Flux

GitOps improves operational consistency significantly.

---

83. How do you troubleshoot failed Helm deployments?

Answer

I start with:

```
helm status
helm history
```

Then validate:

- values.yaml
- template rendering
- Kubernetes events
- RBAC
- hooks

Commands:

```
helm template
helm lint
```

Most failures are:

- invalid values
- resource conflicts
- dependency mismatches

---

84. Explain Blue-Green deployment rollback strategy.

Answer

Rollback is immediate because both environments exist.

Process:

1. Shift traffic to Green
2. Monitor SLIs
3. If failures occur:
4. reroute traffic back to Blue

Benefits:

- fast rollback
- safer upgrades

Challenges:

- database compatibility
- infra duplication cost

Rollback strategy must be tested before production.

---

85. How do you secure container images?

Answer

My strategy:

- minimal base images
- non-root containers
- signed images
- image scanning
- patch management
- immutable tags

Tools:

- Trivy
- Clair
- Cosign

I also restrict:

- untrusted registries
- latest tags

Container security begins at build time.

---

86. How do you handle certificate expiration outages?

Answer

First priority:

Restore service quickly.

Then:

1. Identify expired cert
2. Renew certificate
3. Reload services
4. Validate chain/trust

Long-term prevention:

- automated renewal
- expiration alerts
- cert-manager
- centralized certificate inventory

Certificate automation is essential at scale.

---

87. Explain service mesh and when to use it.

Answer

A service mesh manages service-to-service communication.

Capabilities:

- mTLS
- traffic routing
- retries
- observability
- rate limiting

Popular meshes:

- Istio
- Linkerd
- Consul

I use service mesh when:

- microservices scale significantly
- advanced traffic control needed
- zero-trust networking required

Service meshes add operational complexity, so they should solve real problems.

---

88. How do you debug mTLS failures in Istio?

Answer

Common causes:

- certificate mismatch
- trust issues
- sidecar injection problems
- policy mismatch

I check:

```
istioctl proxy-status
```

Then validate:

- PeerAuthentication
- DestinationRule
- certificate rotation
- Envoy logs

mTLS failures often appear as intermittent connectivity problems.

---

89. Explain circuit breakers and retries.

Answer

Retries improve resilience for transient failures.

Circuit breakers prevent cascading failures.

Without circuit breakers:

- retry storms can overload systems

Good retry design includes:

- exponential backoff
- jitter
- retry limits

Reliability engineering is about controlled failure handling.

---

90. How do you troubleshoot Kafka lag?

Answer

First I identify:

- producer issue?
- consumer issue?
- broker issue?

Checks:

- consumer throughput
- partition imbalance

- broker saturation
- GC pauses
- network latency

Mitigations:

- scale consumers
- rebalance partitions
- optimize batch sizes

Kafka lag directly impacts downstream latency.

---

91. What is your strategy for disaster recovery?

Answer

I define:

- RPO
- RTO

Then design:

- backups
- replication
- failover automation
- multi-region recovery
- restore testing

Most important:

Regular DR drills.

Untested DR plans are unreliable.

---

92. How do you perform RCA after major outages?

Answer

I conduct blameless postmortems.

Structure:

1. Timeline
2. Trigger
3. Contributing factors
4. Detection gaps
5. Recovery analysis
6. Action items

Goal:

Improve systems, not blame individuals.

---

93. Explain immutable infrastructure.

Answer

Immutable infrastructure means servers are never modified manually.

Instead:

- new images built
- new instances deployed
- old ones replaced

Benefits:

- consistency
- reproducibility
- reduced drift
- simpler rollback

This is foundational for reliable cloud operations.

---

94. How do you optimize cloud cost without affecting reliability?

Answer

I optimize carefully.

Strategies:

- rightsizing
- autoscaling
- spot workloads
- reserved instances
- storage lifecycle policies
- efficient resource requests

But I never compromise:

- redundancy
- observability
- recovery capability

Reliability always comes first for critical systems.

---

95. Explain sidecar containers and common use cases.

Answer

Sidecars run alongside the main application container.

Common use cases:

- logging
- monitoring
- proxying
- secret injection

Examples:

- Envoy
- FluentBit
- Vault Agent

Sidecars simplify platform capabilities but increase resource overhead.

---

96. How do you debug slow Docker builds?

Answer

I analyze:

- dependency downloads
- Docker layer invalidation
- unnecessary COPY commands
- image size

Optimizations:

- layer caching
- multi-stage builds
- dependency caching
- minimal context

Small Dockerfile improvements can reduce build time significantly.

---

97. Explain ephemeral containers in Kubernetes.

Answer

Ephemeral containers are temporary debugging containers.

Used for:

- troubleshooting
- inspecting running pods
- debugging minimal containers

Example:

```
kubectl debug
```

Very useful when production containers lack debugging tools.

---

98. How do you handle noisy neighbor problems in Kubernetes?

Answer

Noisy neighbors occur when workloads compete aggressively.

Mitigation:

- requests/limits
- taints/tolerations
- node pools
- quotas
- isolation policies

Resource governance becomes critical in multi-tenant clusters.

---

99. What is the difference between vertical and horizontal scaling?

Answer

Vertical scaling:

- increase CPU/memory of same instance

Horizontal scaling:

- increase number of instances

Horizontal scaling improves:

- availability
- fault tolerance
- elasticity

Most cloud-native systems prioritize horizontal scaling.

---

100. What mindset is required for Staff-level DevOps/SRE roles?

Answer

Staff-level engineers think beyond tooling.

They focus on:

- resilience
- scalability
- platform enablement
- operational excellence
- engineering standards
- incident leadership
- organizational impact

Most importantly:

They design systems assuming failure is inevitable and engineer platforms that recover gracefully.